

THE RECTIFICATION CONTINUES

JSON Consistency Repair as a Recursive Operational Deployment of Solution Causale

Software discussed: json-consistency-repair 0.41.0

Solution Causale: <https://zenodo.org/records/21925174>

DOI: <https://doi.org/10.5281/zenodo.21925174>

Research note - August 21, 2026

Version 0.41.0 is the final verified PASS041 release candidate described by the included source, benchmarks and release-verification artifacts.

Abstract

This document presents json-consistency-repair 0.41.0 in the same publication model used for CSV Consistency Repair: ordinary software at the point of use, explicit research documentation at the point of explanation. The package is not only a malformed-JSON fixer. It discovers stable relations, reconstructs missing or contradicted values when the evidence identifies one terminal, applies minimal reversible patches, remeasures the updated object, tracks authority and provenance, actively tries to falsify learned relations, preserves unresolved obligations, and verifies closure through replay and independent certification.

PASS041 adds a two-mode execution architecture. Full mode performs exhaustive global analysis, replay and certification. Fast mode still scans all analyzer families but restricts expensive deepening to unique local actionable cones and explicitly reports FAST_SCAN_ONLY rather than claiming full certification. The final verification records 462/462 regression tests, a 600/600 adversarial PASS041 stress, 73/73 valid public Draft 2020-12 schemas, reproducible wheel/source builds and fresh-install parity.

The benchmark evidence is deliberately bounded. It establishes exact recovery on declared corpora, not universal superiority over every JSON repair product. The direct local comparison shows a strong advantage on deterministic malformed-JSON recovery and semantic consistency reconstruction, while optimized parsers remain vastly faster on ordinary valid JSON. The remaining external gate is a direct comparison against dedicated third-party JSON-repair systems not installed in the test environment.

Keywords

JSON repair; consistency repair; constraint discovery; minimal reversible correction; provenance; authority; active falsification; abstention; proof replay; bounded-memory streaming; distributed coherence; causal computation; Solution Causale.

Document status

Release-oriented architecture and evidence note. Numerical claims are restricted to the included benchmark corpora and verification artifacts.

Contents

1. Purpose and publication position
2. What the software actually is
3. SC-to-software translation
4. The recursive rectification cycle
5. Why JSON is the next domain
6. Architecture of version 0.41.0
7. Final verification evidence
8. Evidence boundaries under corruption
9. Fast and full execution modes
10. Syntax frontier and ambiguity
11. Authority, symmetry and controllability
12. Falsification, persistent OPEN and distributed coherence
13. Residual information and blind reconstruction
14. Comparative benchmark
15. Current limits and falsifiable claims
16. Reproducibility and release notes
17. Conclusion

1. Purpose and publication position

The software interface remains conventional on purpose. A user can install a JSON repairer, inspect a machine-readable report, run dry mode, process bundles or streams, and choose between fast and full execution without learning SC vocabulary. The research note makes the deeper construction public rather than hiding it.

The separation is functional: ordinary JSON tooling at the interface, explicit causal architecture in the research record.

2. What the software actually is

At the surface, json-consistency-repair is a dependency-free Python package for conservative recursive JSON repair. It covers strict parsing, bounded grammar repair, nested objects and arrays, exact arithmetic, schemas and DSL constraints, cross-document identity, bundles, JSONL/NDJSON and top-level-array streaming, dry-run planning, inverse replay and machine contracts.

Architecturally, it is a constrained rectification engine. A mutation must be supported by an admissible relation or authority source, survive the relevant safety gates, improve the measured state, remain controllable under permissions and budgets, and close under replay. When the terminal is not uniquely determined, the engine preserves the ambiguity.

3. The SC-to-software translation

SC / causal notion	Operational software realization
Object	JSON document, bundle or stream
Invariant	Schema, mapping, arithmetic, identity, temporal or cross-field relation
Residue / incoherence	Violation, contradiction, open obligation or conflicting reconstruction
Admissible displacement	Candidate reversible JSON Patch / structured repair
Minimal correction	Smallest certified state-improving mutation or atomic plan
Authority / provenance	Evidence source and scope that may authorize a conclusion
Closure	Fixed point + replay + proof-graph / certification checks
Open terminal	Explicit abstention or typed missing witness

4. The recursive rectification cycle

- **Observe:** parse the carrier while preserving structure, identity and provenance.
- **Discover:** infer stable relations and compile declared schemas, DSL constraints and source roles.
- **Falsify:** actively attack learned relations before allowing portability or repair authority.
- **Localize:** map violations and open obligations to the smallest relevant causal cone.
- **Reconstruct:** generate candidate terminals only from admissible evidence.
- **Control:** verify permissions, immutable paths, budgets and legal mutation sequences.
- **Commit minimally:** apply the smallest improving reversible patch or atomic plan.
- **Remeasure:** analyze the physically updated object and preserve newly exposed relations.
- **Verify:** check fixed point, replay, inverse consistency, proof graph and certification as required.
- **Abstain:** leave the object unchanged when the evidence does not identify one terminal.

5. Why JSON is the next domain

JSON is structurally richer than CSV: hierarchy, arrays, object identity, nested scope, references, schemas, ordering semantics, multiple documents and streaming boundaries all matter. That makes it a stronger test of whether rectification principles survive beyond a rectangular table.

6. Architecture of version 0.41.0

Layer	Representative modules
Entry / orchestration	cli.py, engine.py, models.py
Constraints / exact math	constraint_ir.py, constraint_dsl.py, exactmath.py, linear_exact.py
Structure / syntax	grammar_repair.py, structural.py, schema_bridge.py, tree.py
Authority / proof	authority_scope.py, provenance.py, provenance_chain.py, proof_graph.py, certifier.py
Decision / minimality	minimal_transfer.py, parallel_routes.py, primary_factorization.py, symmetry.py, controllability.py
Robustness	relation_falsifier.py, persistent_open.py, horizon.py, information_bounds.py
Scale / cross-document	bundle.py, multisource.py, streaming.py, federation.py

7. Final verification evidence

Verification gate	Recorded result
Regression suite	462/462 PASS
PASS041 adversarial stress	600/600; 0 false mutations
Public Draft 2020-12 schemas	73/73 valid
Fresh extracted source	PASS040+041 25/25; stress 600/600; compileall PASS
Fresh wheel	pip check PASS; full + fast exact controlled repair
Reproducible builds	wheel and source rebuilt independently byte-identical

These checks establish release reproducibility and the declared software contract. They do not prove universal correctness on unknown external data.

8. Evidence boundaries under corruption

The limit campaign separated missing information from corrupted information. A learned carrier can fail when the observations no longer identify which coherent regime is intended. That is an identifiability boundary, not automatically a repair-capacity ceiling.

Bound test	Corruption	Exact repairs	False mutations
Functional authoritative implications	39/40 wrong targets (97.5%), 3 seeds	117/117	0
Arithmetic authoritative balance	39/40 wrong totals (97.5%), 3 seeds	117/117	0
Learned functional carrier	20/40 missing values, 3 seeds	60/60	0

9. Fast and full execution modes

Rows	v0.40 full	0.41 full	0.41 fast	fast vs 0.40
100	1.771 s	0.800 s	0.0275 s	64.4x
500	7.715 s	2.555 s	0.124 s	62.1x
1000	16.622 s	4.496 s	0.250 s	66.5x

10. Syntax frontier and ambiguity

The final PASS041 syntax contract records 14/14 correct outcomes: 13/13 determinable corruptions repaired and 1/1 conflicting duplicate-key ambiguity refused. The repaired set includes missing/trailing commas, missing colon, bounded truncation, missing quote, extra brace, two bounded structural errors, single quotes, unquoted keys and comments when the bounded lexical route converges to one strict JSON terminal.

`{"a":1,"a":2}` is not silently resolved. Without authority, two conflicting duplicate values do not identify which occurrence is the intended survivor.

11. Authority, symmetry and controllability

Later passes changed the question from “what value looks plausible?” to “what conclusion is actually authorized and reachable?” Source roles and provenance are tracked; external policies cannot silently claim authority; symmetric alternatives block arbitrary index-based choices; immutable paths, allowed operations, budgets and precedence constraints determine whether an identifiable repair is actually controllable.

The result is a stricter separation among UNKNOWN, identifiable-but-unreachable, reachable and controllable states. A mathematically known target is not treated as applicable unless a legal mutation sequence exists.

12. Falsification, persistent OPEN and distributed coherence

Learned relations are actively attacked by held-out checks, negative controls, ablations and counterexample search before promotion. Relations that fail are firewalled before federation and optimization. Unresolved problems persist as typed OPEN obligations such as missing source, missing permission, missing symmetry breaker or missing rewrite join; they can wake only when relevant evidence arrives.

Incremental recomputation is compared against fresh full recomputation. Horizon/distributed checks ensure that a local conclusion does not drift merely because a document, shard set or streaming window was enlarged; unexplained drift forces full recomputation or conflict rather than silent mutation.

13. Residual information and blind reconstruction

PASS040 makes information insufficiency measurable. For a finite set of n remaining candidates, the engine records the exact selection requirement $\text{ceil}(\log_2(n))$ when the candidate cardinality is established; it does not fabricate a zero-bit bound when the carrier is unknown. Blind reconstruction physically removes the target before inference and verifies the prediction only afterward against a committed digest.

14. Comparative benchmark

Tool	Exact recoveries	Wrong accepted	Rejected
JCR grammar repair	200/200	0	0
JSON5	40/200	0	160
PyYAML safe	40/200	40/200	120
stdlib json	0/200	0	200
orjson	0/200	0	200
ujson	0/200	0	200

On the shared malformed-JSON corpus, JCR recovered all 200 uniquely recoverable targets. The same benchmark does not convert parser-only tools into semantic repairers; cross-record reconstruction, authoritative correction, replay and abstention are treated as capability differences rather than artificial 0% scores.

Performance remains asymmetric. On a 7,711-byte valid document, orjson measured about 271 MB/s, ujson 220 MB/s and stdlib json 134 MB/s, while JCR grammar exact measured 8.48 MB/s and the full consistency engine about 0.0112 MB/s. JCR is not a raw parsing-speed competitor.

15. Current limits and falsifiable claims

- A direct benchmark against dedicated third-party JSON-repair systems remains external.
- Heavy corruption without an independent bound can remain genuinely underdetermined.
- Fast mode is not full certification; it advertises FAST_SCAN_ONLY.
- PASS is contract-relative: no unresolved violation under certified relations, not inaccessible external ground truth.
- Resource/security limits deliberately refuse evidence beyond configured bounds rather than truncating silently.

16. Reproducibility and release notes

The release package includes source, wheel, tests, schemas, human/LLM guides, machine contract, raw benchmark results, scripts, historical pass reports and final verification. The wheel and source are reproducible byte-for-byte under the release builder; fresh-source and fresh-wheel checks are part of the release evidence.

17. Conclusion

json-consistency-repair 0.41.0 should be understood as the JSON deployment of the same rectification program first made operationally visible in CSV, but on a carrier with substantially richer structure. The central object is not a permissive parser. It is a cycle that observes, discovers, falsifies, localizes, reconstructs, controls, commits minimally, remeasures and verifies.

The development sequence progressively removed shortcuts that would have made the result easy to overstate: provenance can block authority; symmetry can block arbitrary choices; a known target can still be unreachable; learned relations are actively challenged; unresolved obligations remain explicit; incremental results are checked against full recomputation; horizon changes must carry an explanatory witness; residual information is bounded; blind reconstruction prevents target leakage; and fast mode never presents itself as full certification.

The strongest defensible evidence statement is therefore narrow but substantial: on the included deterministic malformed-JSON corpus, the grammar repair layer recovered 200/200 targets with no wrong materializations; with strong independent bounds, the tested functional and arithmetic families repaired 97.5% corrupted targets exactly with zero false mutations; the final package passes its complete recorded release-verification gates; and the fast path provides large speedups on the measured repair corpus while preserving the same exact terminal. None of these local results implies universal superiority over all JSON repair systems.

The public claim is not “JSON is solved.” The claim is that a reproducible rectification engine now exists whose decisions, abstentions, proof scope and remaining external comparison gates are explicit and testable.

Release coordinates

Software: json-consistency-repair 0.41.0

Python: ≥ 3.10

License: MIT

CLI: json-consistency-repair

Solution Causale: <https://zenodo.org/records/21925174>

DOI: <https://doi.org/10.5281/zenodo.21925174>

Construction note

The construction of json-consistency-repair was completed with GPT. Solution Causale: <https://zenodo.org/records/21925174> - DOI: <https://doi.org/10.5281/zenodo.21925174>